Building a CNN for recognizing garbage images

Project for the "Algorithms for Massive Data" course

Franco Faúndez Cabion - 48085A

Data Science for Economics

Università degli Studi di Milano

June 14, 2026

Contents

1 Introduction **1**

2 Methodology **2**

3 Data exploration **2**

4 Preprocessing **5**

5 Modeling **6**

5.1 Experimental setup 6

5.2 Baseline model 7

5.3 MobileNetV2 9

6 Results **10**

7 Conclusion **13**

1 Introduction

Over the past 20 years, convolutional neural network (CNN) models have evolved, becoming increasingly sophisticated and complex. Architectures such as ResNet and EfficientNet demonstrate the effectiveness of CNNs and position them as state of the art methods for image classification tasks.

Despite their strong performance, reaching such results typically requires very large datasets for both training and tuning. As the availability of data continues to grow and tasks become more specific, models tend to become larger and more complex. While this often improves performance, it also increases computational requirements and hence, scalability, making these models impractical when it comes to training large datasets.

To address this challenge, one possible approach is to design a smaller CNN architecture capable of processing large image inputs efficiently. Such models can significantly reduce computational cost and training time, but unlikely to achieve the same level of performance as larger, complex models on more challenging classification tasks.

Conversely, training large CNN models often leads to better accuracy particularly when more data is available for training. Furthermore, the learned weights of previously trained CNNs can be reused for different tasks, allowing models to leverage knowledge acquired from larger datasets. Nevertheless, these models typically require substantially longer training times and greater computational resources.

The primary objective of this work is to analyze the tradeoff between model performance and computational efficiency when classifying garbage images, using this dataset as a theoretical proxy for larger scale applications.

NOTE: "I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study"

2 Methodology

First, exploration of the images will be conducted to understand their features and determine the level processing needed before training. A file quality check will be performed to identify any corrupted or invalid images within the dataset, and treated properly if necessary. This phase will also measure how well distributed labels are, which is an important aspect to consider before modeling.

Based on these findings, the data will then be prepared for model training. This step is essential since it defines the input for all models that will be implemented. Among the techniques considered, there is image resizing, batch size configuration and image normalization. Subsequently, the dataset will be dividied into training, validation and test sets.

Next, hyperparameter tuning will be carried out for both models using the validation set. Based on these results, the optimal parameters from the given tuning search space will be selected, and each model will be trained with these on the full training dataset.

Finally, model evaluation will be conducted using the test set. Classification metrics will be used to assess predictive performance and generalization ability. Given the assumption that this dataset could grow to a much larger size, the final step of the analysis will be to compare which implementation would scale better with more data. Models will be initially developed, tested and evaluated in a local environment before being migrating an executable copy of them to Google Colab.

3 Data exploration

The dataset consists of 13348 images representing different types of garbage: glass (1998), clothes (1984), plastic (1709), shoes (1633), cardboard (1533), paper (1380), metal (993), battery (814), biological (810) and trash (494). These images vary considerable in terms of pixel width and height, Figure 1 shows the distribution of sizes across the dataset. Since convolutional neural networks require an input with consistent dimensions, all images must be resized the same in the preprocessing phase (except for MobileNetV2, which will be covered later).

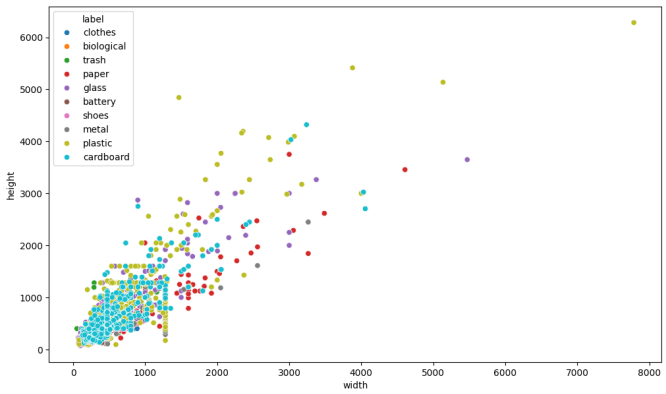

Figure 1: Distribution of image sizes for different labels

Figure 2 shows a sample of images from the dataset. We can see that keeping a three dimensional color scale can be useful for the models to retain aspect of depth and color, which can give information at the moment of distinguishing materials and shapes.

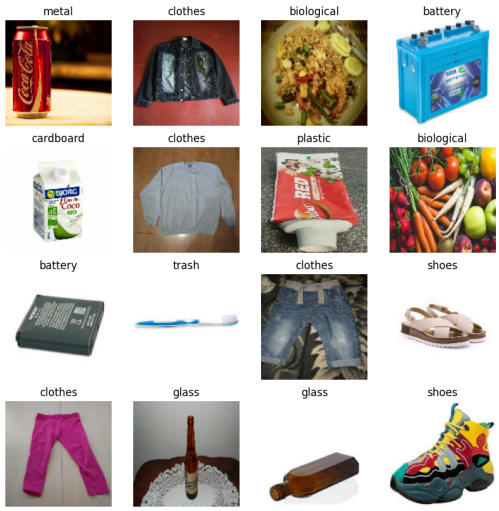

Figure 2: Labeled sample of images from the dataset

Further examination reveals some aspects that can be challenging for the models. For instance, some images labeled as cardboard correspond to printed packaging such as juice and food boxes, which contains graphics that can allude to other labels (i.e. food). In such cases, the model might incorrectly associate these visual elements in classes that don't directly correspond to the target. In contrast, other

images are more consistent, for example showing plain brown boxes which may be easier for the model to identify.

Figure 3: Small sample of cardboard images

A similar issue could happen to both metal and plastic labels, where certain colors and shapes may resemble objects typically associated with glass. In this sense, paper category may also overlap visually with other classes such as cardboard and other categories, since they correspond to magazines or printed materials.

Figure 4: Small sample of paper images

Other categories appear to present fewer ambiguities, such as glass, battery or clothes, since they show elements that are generally more distinctive. A particular case is the category trash: many images in this class are face masks or toothbrushes, while fewer examples actually represent miscellaneous trash items. As a result, the model may struggle to correctly identify other forms of trash that are underrepresented.

Finally, all images exhibit differences in orientation, scale, shapes and zoom levels within the same category. This diversity is highly beneficial for model training, as including different versions of the same category encourages models to generalize better. This effect is also achieved through data augmentation [1], although here would not be necessary.

4 Preprocessing

As covered during exploration of the dataset, several preprocessing steps were considered before the modeling phase.

First, to determine the image resolution of the input a test was done to compare time efficiency during training, since one of the main goals is to obtain scalable results. Several runs were done comparing execution time for training, where a 32x32 pixel shape resulted to be the most efficient without losing too much performance over the baseline model. This resolution was chosen only for the latter model, since the MobileNetV2 model requires to set the shape at least at 96x96 [2].

In addition to resizing, pixel normalization was applied to improve numerical stability during training. This has proven to lead into faster convergence while training [3], at the same time improving model performance and scaling better with more data.

Converting images from red-green-blue (RGB) color model to grayscale was also considered since the latter reduces computational complexity by compressing information into a single channel instead of three [4]. However, exploratory analysis suggests that color has a significant impact at detecting shapes by more complex depth understanding. Therefore, RGB representation was kept as the standard for all models.

Similarly, data augmentation techniques were evaluated but ultimately not implemented. While this can improve model generalization by increasing dataset variability, the current dataset already contains a wide level of diversity. Since the primary objective of the project is scalability rather than maximizing predictive performance, introducing additional steps would increase computational steps without providing a substantial benefit within the main objective.

Finally, the dataset was divided into training, validation, and test sets, with proportions of 72%, 13%, and 15%, respectively. The validation set is used to monitor model performance during training and hyperparameter selection, while the test set is kept for the final evaluation of the model. Although cross-validation was initially considered for hyperparameter tuning, it was ultimately discarded due to the computational cost associated, hence not scaling well at bigger datasets.

The batch size was set to 64, balancing training efficiency and memory constraints. This value allows the model to benefit from stable gradient estimates while main-

taining reasonable training times [5].

5 Modeling

5.1 Experimental setup

Before training the models, hyperparameter tuning was performed for both implementations. The holdout validation set was used to evaluate parameter combinations within a small tuning space, where the selection criteria of optimal parameters is based on the lowest validation loss.

Tuning ranges were defined based on widely adopted values from common practice while at the same time ensuring that the search space remained limited to avoid excessive computational cost. This methodology applies for kernel size, number of neurons and optimization-related parameters. Regarding this last point, Adam was used as the optimizer for both models due to its high adaptability and robust performance across a wide range of tasks and data sizes [4].

Parameter	Value
Learning rate	[1e-2, 1e-3]
Kernel size	[3, 5]
Beta 2 (exponential decay)	[0.99, 0.999]
Number of filters	[64, 128]
Number of neurons	[64, 128]

Table 1: Tuning space for baseline model

Parameter	Value
Learning rate	[1e-2, 1e-3]
Beta 1 (momentum)	[0.0, 0.9]
Beta 2 (exponential decay)	[0.99, 0.999]

Table 2: Tuning space for MobileNetV2

It is worth to mention that unlike the dataset's partition, each training process is stochastic due to random weight initialization at every run. This implies a positive effect since the performance analysis involved training each architecture fifteen times to measure general behavior with optimal parameters, giving a closer performance metric with statistical reliability.

Early Stopping is used to narrow execution time when validation loss does not improve for 5 consecutive epochs (patience parameter). The number of epochs are different depending on the task: for tuning are 15 while for training are 30.

5.2 Baseline model

The first model is a simple baseline implementation consisting of two convolutional blocks, each composed of a convolutional layer followed by a max pooling layer. Then, a global average pooling layer is applied to aggregate information produced by the convolutional layers before passing the result to a dense layer with dropout. This pooling technique is used instead of a traditional flatten operation, as flattening the feature maps would significantly increase the number of parameters inside fully connected layers and consequently lead to a higher risk of overfitting, especially given the relatively simple architecture and dataset size.

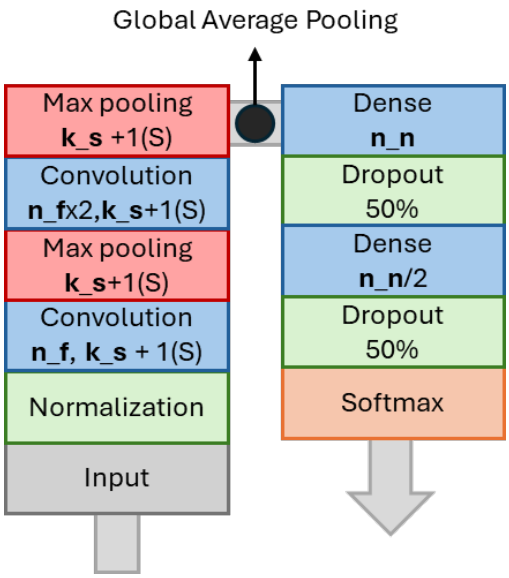

Figure 5: Structure of custom architecture

For clarity, n_f denotes the number of filters, k_s the kernel size of the convolutional layers, and n_n the number of neurons in the dense layer. As shown in Figure 5, the number of filters increases across convolutional layers, allowing the network to capture progressively more complex features down for the dense layer. The final output layer contains 10 neurons corresponding to the number of classes, where a softmax activation function is applied to generate class probabilities and the predicted class is obtained by selecting the one with highest score.

The final architecture selected through hyperparameter tuning uses n_f = 128 filters, k_s = 5x5 kernels, n_n = 128 neurons in the dense layer, while a learning rate of 1e-3 and Beta 2 = 0.99 was selected for the optimizer. The format used to represent this architecture was inspired by the notation presented in the work of Aurélien Géron [4], which provides a clear and intuitive way to visualize neural network structures.

Performance is analyzed as an aggregate metric, in this case accuracy and loss for both training and validation over 15 runs of the same model using optimal hyperparameters. Regarding the baseline model, at earlier epochs both training and validation metrics are close to each other, although validation performance is often slightly better. This can be explained by the use of dropout in the architecture since it acts as a regularization technique by randomly deactivating a fraction of neurons during training, making learning more "challenging" for the model. Consequently, the model's performance on the training set is reduced compared to validation, where dropout is disabled and full network is used, often leading to better metrics.

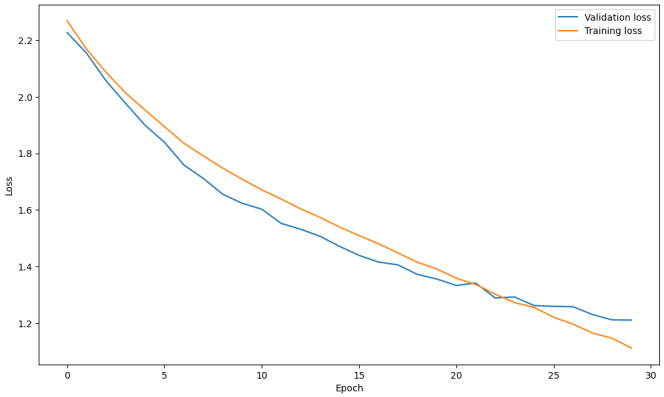

Figure 6: Validation and training loss evolution for baseline model

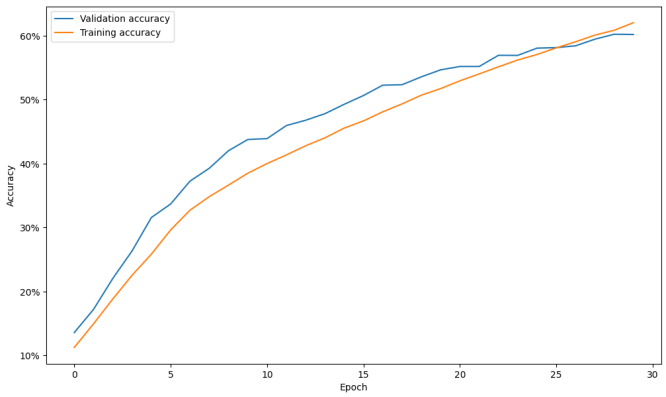

Figure 7: Validation and training accuracy evolution for baseline model

5.3 MobileNetV2

The second model is a pretrained MobileNetV2 model, initialized using ImageNet weights. This model uses linear bottlenecks, where features are temporarily expanded and processed with efficient depthwise separable convolutions and then compressed back to a low dimensional representation while preserving information through residual connections [2]. Compared to traditional CNN architectures such as VGGNet or ResNet, MobileNetV2 has a reduced number of parameters while maintaining decent accuracy, being suitable for lightweight implementations on reduced memory systems.

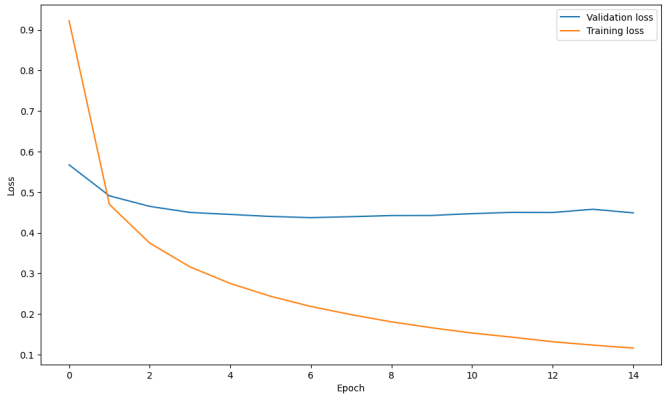

Figure 8: Validation and training loss evolution for MobileNet

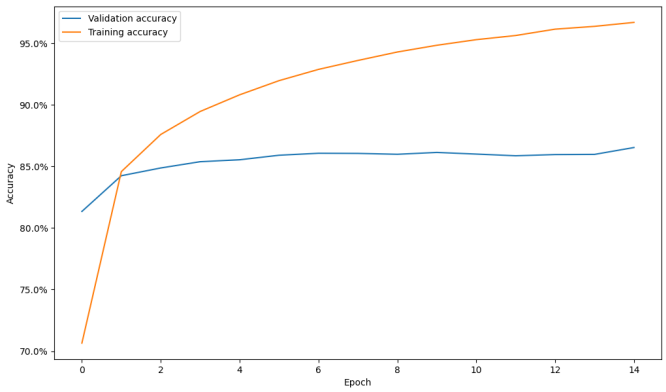

Figure 9: Validation and training accuracy evolution for MobileNet

The performance of this model is higher compared to the baseline, with both training and validation metrics showing improved values. However, the model exhibits clear signs of overfitting. This is reflected in the loss and accuracy curves, where a pleateau is reached very early at 0.47 and 85% for both, respectively. This creates a growing gap, as shown in Figure 9.

6 Results

Evaluation on the test set provides insight into how the models perform on unseen data. For the baseline model, the accuracy was 58% and the weighted F1-score 56%. Weighted metrics were used since, it appropriately accounts for the importance of each class, especially in this highly imbalanced dataset. Considering the simplicity of this architecture and the relatively small size of the dataset, these results represent a reasonable baseline for comparison with more complex models. Being lightweight and not overfitted, this model could serve as a minimal initial solution for this task.

For the second model, results on the test set are more promising, achieving an accuracy of 86% and a weighted f1-score of 86%. These results are strong, and despite exhibiting clear signs of overfitting through a final training accuracy of 96%, the model remains as best choice for this dataset. Moreover, if the size of the dataset were to increase it is likely that the model's generalization performance would improve, as additional data could help mitigate overfitting. Given its higher complexity, the model is better at adapting to larger and more diverse datasets in comparison to the baseline model.

The comparison of both confusion matrices indicates that the second model achieves better overall performance, as it produces fewer misclassifications and shows a clearer separation between most classes.

In contrast, the baseline model exhibits more frequent errors, particularly between categories that are visually similar. For example, a significant number of shoe items are misclassified as clothing. Similar patterns appear with batteries being predicted as metal, glass as plastic, and paper as cardboard, where overlapping visual features such as texture, shape, or color contribute to the confusion. However, a notable inconsistency is the misclassification of materials like cardboard, metal, or glass as biological label. This type of error is less intuitive, although it may be partially explained by the presence of printed organic item on some items (cereal box, juice box, etc.).

The second model follows a similar pattern of errors but with a substantially lower frequency. Its most common misclassifications include predicting paper as cardboard and confusing glass with plastic or metal, which are reasonable given their similar visual characteristics. Additionally, the model occasionally labels general trash items as plastic, likely due to the presence of everyday objects such as toothbrushes that make the distinction between these categories less clear. Overall, while both models share similar types of confusion, the second model demonstrates clear improved accuracy and consistency.

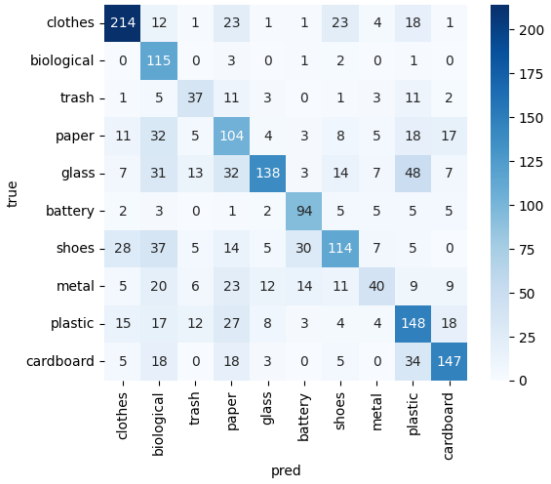

Figure 10: Confusion matrix for results using baseline model

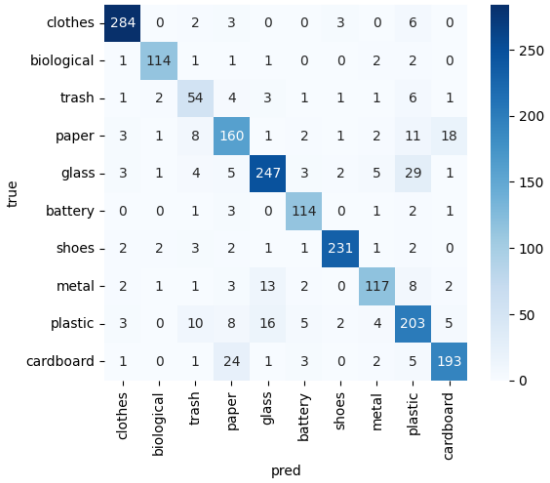

Figure 11: Confusion matrix for results using MobileNetV2

7 Conclusion

First, the model that achieved the best performance in terms of classification metrics was MobileNetV2. This can be mainly attributed to two factors. On one hand, this model is inherently more complex than the baseline model, which consists of a limited number of convolutional and dense layers. MobileNet requires a larger input size, which provides it with significantly richer visual information. Combined with its architectural features, such as a reduced number of parameters compared to wider models and the use of linear bottlenecks to preserve relevant information, makes this model better at generalization and overall performance in the classification task, even with detected overfitting.

The second key factor is that MobileNet is a pre-trained model: its weights, initially learned on large dataset, can be fined tuned for the specific task at hand. This transfer learning approach allows the model to leverage previously learned visual patterns, making it more effective at generalizing and recognizing features that a model trained from scratch might fail to capture or would take more time and data to reach it.

Beyond achieving superior performance, MobileNet also demonstrates that it scales better with increasing data. Training a model from scratch can be computationally expensive, even if its relatively small as this project covered. In contrast, using a pre-trained model allows building on top of already learned representations, significantly reducing training effort. As this dataset grows, this model would be less prone to overfitting, closing the gap between training performance and generalization. An additional advantage is that it remains lightweight compared to more complex architectures (such as VGGNet or ResNet), offering a strong balance between performance and computational efficiency.

Other design choices also contributed to performance and scalability. In this sense, selecting a lower but reasonable image resolution for the baseline model improved execution time while maintaining acceptable accuracy, which is beneficial for scalability. Similarly, preserving RGB color formatting instead of converting to grayscale proved important as color information provides visual cues that enhance the model's ability to learn discriminative features.

Finally, hyperparameter tuning plays a crucial role in achieving an efficient model. Tuning space was intentionally limited to reduce computational cost. However, this approach may not scale well for larger datasets or more complex architectures. More efficient optimization techniques, such as Bayesian optimization could improve this process. This represents an opportunity for future improvements.

References

[1] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012. URL: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.

[2] Mark Sandler et al. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. 2019. arXiv: 1801.04381 [cs.CV]. URL: https://arxiv.org/abs/1801.04381.

[3] Yann Lecun et al. "Efficient BackProp". In: (Aug. 2000).

[4] Aurlien Geron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 3rd. Beijing: O'Reilly Media, 2023. ISBN: 978-1098125974.

[5] Samuel L. Smith et al. *Don't Decay the Learning Rate, Increase the Batch Size*. 2018. arXiv: 1711.00489 [cs.LG]. URL: https://arxiv.org/abs/1711.00489.